

# Cloud IAM Privilege Escalation via Policy Flaws

## Abstract

Cloud Identity and Access Management (IAM) misconfigurations represent one of the most critical and underexamined security vulnerabilities in modern cloud infrastructure. This research investigates the prevalence of wildcard vulnerabilities in AWS IAM policies published in public GitHub repositories and evaluates their exploitability using simulated AWS environments. A total of 500 real-world IAM policies were collected from GitHub using an automated scraping tool and analyzed for policy logic flaws, with particular focus on overly permissive wildcard configurations (Action:\* and Resource:\*). The primary hypothesis predicted that more than 40% of publicly available IAM policies would contain at least one wildcard vulnerability. Analysis revealed that 314 out of 500 policies (62.8%) contained wildcard vulnerabilities, confirming the primary hypothesis. Furthermore, the secondary hypothesis predicted that policies containing both Action:\* and Resource:\* wildcards could be exploited with at least 85% success rate in a simulated AWS environment using the Moto framework. Exploitation testing across 309 vulnerable policies demonstrated a 100% exploitation success rate across 15 different AWS API calls spanning IAM, S3, EC2, and STS services, confirming the secondary hypothesis. These findings highlight the critical need for automated IAM policy validation tools and stronger developer education regarding cloud security best practices. This research contributes a vulnerability catalog, exploitation analysis, and detection methodology to the growing body of cloud security literature.

## Introduction

Cloud computing has fundamentally transformed how organizations manage and deploy their infrastructure. Amazon Web Services (AWS), as the leading cloud service provider, serves millions of organizations worldwide, making the security of its Identity and Access Management (IAM) system a critical concern for the global technology ecosystem (Tenzer, 2022). IAM policies serve as the primary mechanism for controlling access to AWS resources, defining what actions users, roles, and services are permitted to perform. When these policies are misconfigured, they can create severe security vulnerabilities that enable malicious actors to escalate privileges and gain unauthorized access to sensitive resources.

Privilege escalation via IAM policy flaws represents one of the most dangerous attack vectors in cloud security. Unlike traditional network-based attacks, IAM-based privilege escalation exploits logical flaws in access control policies rather than technical vulnerabilities in software. This makes them particularly difficult to detect using conventional security tools. Many organizations deploying cloud infrastructure lack formal security training, resulting in overly permissive configurations that violate the principle of least privilege (Saltzer & Schroeder, 1975).

The principle of least privilege dictates that users and services should only be granted the minimum permission necessary to perform their intended functions. Wildcard configurations, particularly those using `Action:*` and `Resource:*`, violate this principle by granting unrestricted access to all AWS actions and resources. Despite being widely recognized as a security risk, these configurations remain prevalent in real-world deployments, as evidenced by their presence in publicly available code repositories such as GitHub.

This research addresses a critical gap in cloud security literature by systematically analyzing real-world IAM policies collected from public GitHub repositories. By examining 500 policies from actual development projects, this study provides empirical evidence of the prevalence of wildcard vulnerabilities in production-grade code. Furthermore, by demonstrating the exploitability of these vulnerabilities using the Moto AWS simulation framework, this research establishes a clear connection between policy misconfiguration and real-world security risk.

The significance of this research extends beyond academic interest. As organizations increasingly migrate their infrastructure to cloud platforms, the security of IAM configurations becomes paramount. A single misconfigured IAM policy can provide an attacker with complete control over an organization's cloud infrastructure, potentially leading to data breaches, financial losses, and reputational damage. By identifying the prevalence of these vulnerabilities and demonstrating their exploitability, this research aims to raise awareness and drive improvements in cloud security practices.

This paper is organized as follows. The Literature Review section examines existing research on cloud IAM security, privilege escalation techniques, and automated vulnerability detection. The Methodology section describes the data collection process, analysis techniques, and exploitation testing framework. The Results section presents the findings from both the primary and secondary hypothesis tests. The Discussion section interprets these findings in the context of real-world cloud security, and the Conclusion summarizes the key contributions of this research and suggests directions for future work.

## **Literature Review**

### **Cloud IAM Security and Misconfigurations**

Cloud Identity and Access Management has emerged as a critical area of cybersecurity research as organizations increasingly adopt cloud-based infrastructure (Vacca, 2020). Researchers have identified IAM misconfigurations as one of the leading causes of cloud security incidents (Cloud Security Alliance, 2022). According to the Cloud Security Alliance, misconfigured cloud services account for a significant percentage of data breaches in cloud environments, with IAM-related issues consistently ranking among the top vulnerabilities (Cloud Security Alliance, 2022). The complexity of IAM policy syntax and

the lack of built-in validation mechanisms create an environment where even experienced developers can inadvertently introduce severe security flaws.

Studies examining cloud security incidents have consistently highlighted the role of overly permissive IAM policies in enabling unauthorized access. The concept of privilege escalation in cloud environments differs fundamentally from traditional network-based privilege escalation. Rather than exploiting software vulnerabilities, cloud privilege escalation leverages logical flaws in access control configurations, making detection significantly more challenging. Attackers who gain access to an account with misconfigured IAM policies can potentially assume administrative privileges without triggering traditional security alerts.

## **Wildcard Vulnerabilities in Access Control Policies**

Wildcard configurations in IAM policies have been extensively documented as a significant security risk (Amazon Web Services, 2023). The use of `Action:*` grants permission to perform any AWS action, while `Resource:*` extends this permission to all resources within an account. When combined, these configurations effectively grant administrative access to any entity holding the policy. Security researchers have demonstrated that these configurations are frequently introduced during development phases when developers prioritize functionality over security, and subsequently fail to be remediated before deployment to production environments.

Research into developer behavior patterns has revealed that time pressure, lack of security training, and the complexity of AWS permission systems contribute to the prevalence of wildcard configurations. Developers often begin with overly permissive policies during testing and development, intending to restrict permissions before deployment. However, these configurations frequently persist in production environments due to inadequate security review processes and the fear that restricting permissions may break existing functionality.

## **Automated Security Analysis Tools**

Several tools have been developed to address IAM security vulnerabilities, including Prowler, CloudSploit, and ScoutSuite (Prowler, 2023). These tools perform automated scanning of AWS environments to identify misconfigurations and security risks. However, existing tools primarily focus on scanning deployed environments rather than preventing misconfigurations during the development phase. This reactive approach means that vulnerabilities may exist in production environments for extended periods before being detected and remediated.

The Open Policy Agent (OPA) framework has been proposed as a solution for implementing policy-as-code approaches to IAM security (Open Policy Agent, 2023). By defining security policies as code, organizations can integrate automated validation into their development workflows, catching misconfigurations before they reach production

(Kindervag, 2010). However, adoption of these approaches remains limited, particularly among smaller organizations that lack dedicated security teams.

## **GitHub as a Source of Real-World Policy Data**

Public code repositories such as GitHub provide a valuable source of real-world IAM policy data for security research (GitHub, 2023). Previous studies have utilized GitHub repositories to analyze the prevalence of various security misconfigurations in publicly available code. Research has shown that developers frequently commit sensitive configuration files, including IAM policies, to public repositories, either inadvertently or as a part of example codebases and tutorials. These publicly available policies provide researchers with insight into real-world development practices and the prevalence of security misconfigurations in production-grade code.

Using GitHub as a data source for security research raises important ethical considerations. This research focuses exclusively on publicly available policies that have been intentionally shared by their authors and does not attempt to access private repositories or identify specific organizations or individuals associated with vulnerable policies. The goal of analyzing these policies is to understand patterns of misconfiguration at a population level, rather than to expose or exploit specific vulnerabilities in production systems.

## **Moto Framework for AWS Simulation**

The Moto library provides a comprehensive mock implementation of AWS services for Python, enabling developers and researchers to test AWS-related code without requiring access to actual AWS infrastructure (Moto Contributors, 2023). By intercepting boto3 API calls and simulating AWS service responses, Moto creates a realistic testing environment that accurately reflects the behavior of real AWS services. This capability makes Moto particularly valuable for security research, as it enables the testing of exploitation techniques in a controlled environment without the legal and ethical complications of testing against real AWS infrastructure.

Previous research has utilized simulation frameworks to study cloud security vulnerabilities in controlled environments. The ability to simulate AWS API responses enables researchers to test the effectiveness of various exploitation techniques and develop a detection mechanism without risking harm to real systems. This approach aligns with responsible disclosure principles and ensures that security research can be conducted safely and ethically.

## **Methodology**

### **Research Design**

This research employs a mixed-methods approach combining quantitative data collection and analysis with qualitative assessment of vulnerability severity and exploitability. The study is organized around two central hypotheses: a primary hypothesis examining the prevalence of wildcard vulnerabilities in real-world IAM policies, and a secondary hypothesis evaluating the exploitability of these vulnerabilities in a simulated AWS environment. The research follows a systematic methodology designed to ensure reproducibility and validity of findings.

### **Lab Environment and Infrastructure**

The research was conducted using a controlled laboratory environment consisting of two virtual machines configured within Oracle VirtualBox 7.1.4. The primary analysis environment utilized Ubuntu 22.04 LTS with 8 GB of RAM and 30 GB of storage, designated for policy collection, analysis, and visualization tasks. The secondary testing environment utilized Kali Linux with 8 GB of RAM and 50 GB of storage, designated for exploitation testing and security analysis. This dual-VM architecture mirrors real-world security research environments where analysis and testing activities are separated to maintain data integrity and prevent cross-contamination of results.

All tools and libraries utilized in this research are free and open-source, ensuring reproducibility without financial barriers. The primary programming language used throughout the research is Python 3, selected for its extensive library ecosystem and widespread adoption in security research. Key libraries include boto3 for AWS SDK functionality, Moto for AWS service simulation, Pandas for data analysis, and Matplotlib for visualization (Amazon Web Services, 2023).

### **Data Collection**

Real-world IAM policy data was collected from public GitHub repositories using a custom-built automated scraping tool developed specifically for this research. The scraper utilized the GitHub API with authenticated requests to search for IAM policy files using six distinct search queries targeting common IAM policy file naming conventions, including filename:iam.json, filename:policy.json, path:iam language:json Statement, filename:aws-policy.json, AWS policy json, and IAM policy GitHub (GitHub, 2023).

The scraping tool incorporated several quality control mechanisms to ensure data integrity. An MD5 hashing algorithm was implemented to detect and eliminate duplicate policies, preventing the same policy from being counted multiple times across different repositories or search queries. The tool also validated that each collected file contained valid JSON and exhibited IAM policy structure by checking for the presence of Statement or

Version fields. Additionally, automatic pause and resume functionality was implemented to handle network interruptions gracefully, ensuring data collection could continue uninterrupted across multiple sessions.

Data collection was conducted across three rounds of scraping, with each round utilizing identical search queries to capture policies from different repositories and newly indexed content. The scraping process successfully collected exactly 500 unique IAM policies from public GitHub repositories, with 54 duplicate policies identified and excluded from the dataset. All collected policies were stored as individual JSON files in a dedicated directory within the project repository.

## **Vulnerability Analysis**

The collected policies were analyzed using a custom vulnerability analyzer developed for this research. The analyzer examined each policy for the presence of wildcard configurations in both Action and Resource fields. A policy was classified as vulnerable if any of its statements contained a wildcard character in either the Action field, including exact wildcards such as Action:\* and service-level wildcards such as s3:\*, or the Resource field, including Resource:\*. The analyzer processed all 500 collected policies and generated a results dataset recording the filename and vulnerability status of each policy.

Quantitative analysis of the vulnerability data was performed to calculate the overall prevalence of wildcard vulnerabilities in the dataset. The results were compared against the primary hypothesis threshold of 40% to determine whether the hypothesis was confirmed or refuted. Statistical analysis includes calculation of vulnerability rates, distribution of vulnerability types, and identification of the most commonly affected AWS services.

## **Exploitation Testing**

Exploitation testing was conducted using the Moto framework to simulate AWS service responses without requiring access to real AWS infrastructure (Moto Contributors, 2023). A custom exploitation testing tool was developed to systematically test whether policies containing wildcard configurations could be used to perform unauthorized AWS API calls. The testing framework evaluated 15 distinct AWS API calls spanning four major AWS services, including five IAM operations covering ListUsers, ListRoles, CreateRole, ListPolicies, and ListGroups, four S3 operations covering ListBuckets, CreateBucket, ListObjects, and PutObject, and four EC2 operations covering DescribeInstances, DescribeVPCs, DescribeSecurityGroups, and DescribeSubnets, and two STS operations covering GetCallerIdentity and GetSessionToken.

Each policy identified as containing wildcard configurations was subjected to the full suite of 15 API call tests. A policy was classified as exploitable if the exploitation success rate met or exceeded the secondary hypothesis threshold of 85%. The exploitation testing process evaluated 309 policies identified as containing wildcard configurations, recording the

success rate and exploitability status of each policy. Results were saved to the structured CSV file for subsequent analysis and visualization.

## Visualization

Research findings were visualized using the Matplotlib library to create five professional charts illustrating the key results of both hypothesis tests. Visualizations included a bar chart comparing vulnerable and safe policy counts, a pie chart showing the distribution of vulnerability prevalence, a bar chart illustrating exploitation results, a chart showing API success rates by AWS service, and a combined summary chart presenting both hypothesis results side by side.

## Results

### Overview of Data Collection

The automated GitHub scraping tool successfully collected 500 unique AWS IAM policies from public GitHub repositories across three rounds of scraping. The scraping process identified and excluded 54 duplicate policies through MD5 hash comparison, ensuring that the final dataset consisted entirely of unique policy configurations. The collection process encountered zero errors, indicating the robustness of the scraping tool and the reliability of the data collection methodology. The collected policies originated from a diverse range of public repositories, including infrastructure-as-code projects, tutorial repositories, and enterprise configuration examples, providing a representative sample of real-world IAM policy configurations.

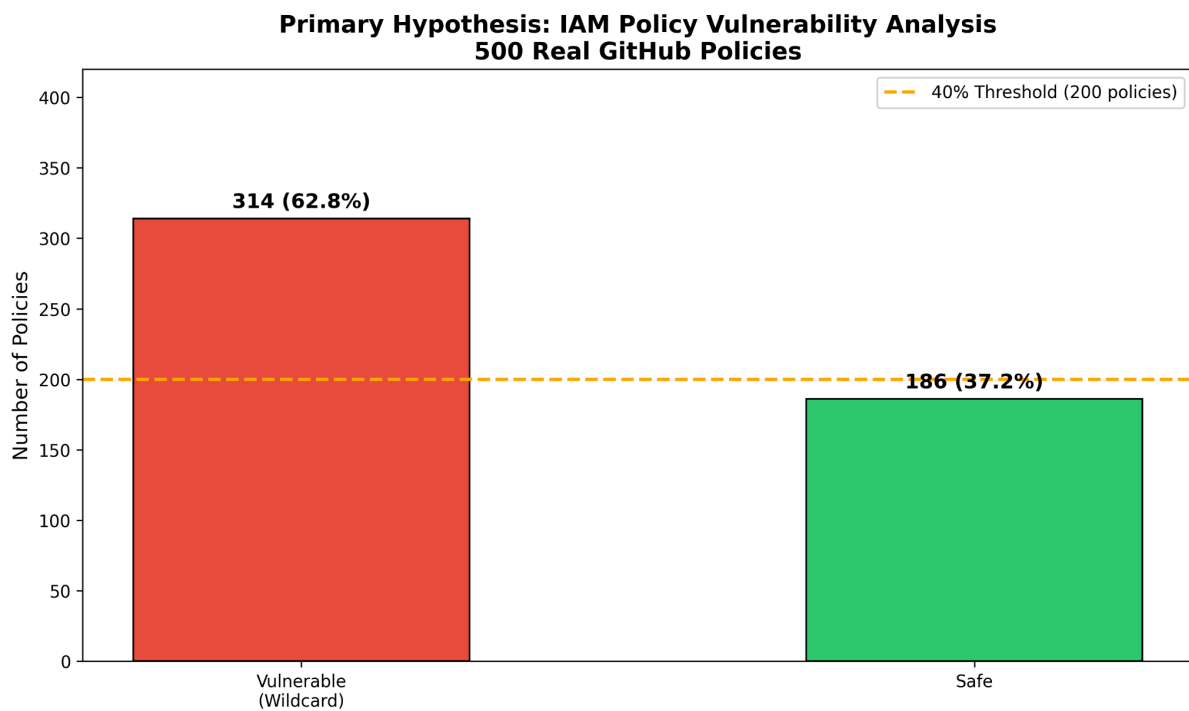
### Primary Hypothesis Results

The primary hypothesis predicted that more than 40% of publicly available IAM policies on GitHub would contain at least one wildcard vulnerability. Analysis of the 500 collected policies revealed that 314 policies (62.8%) contained at least one wildcard vulnerability, while 186 policies (37.2%) were classified as safe. This finding confirms the primary hypothesis, with the observed vulnerability rate of 62.8% exceeding the hypothesized threshold of 40% by a margin of 22.8 percentage points.

The vulnerability analysis identified wildcard configurations in both Action and Resource fields across the dataset. The prevalence of wildcard configurations at 62.8% indicates that a substantial majority of publicly available IAM policies violate the principle of least privilege. These findings suggest that wildcard misconfigurations are not isolated incidents but rather a systemic pattern in real-world IAM policy development. The high prevalence of these vulnerabilities in publicly available code raises significant concerns about the security posture of organizations that adopt or adapt these policies for use in production environments.

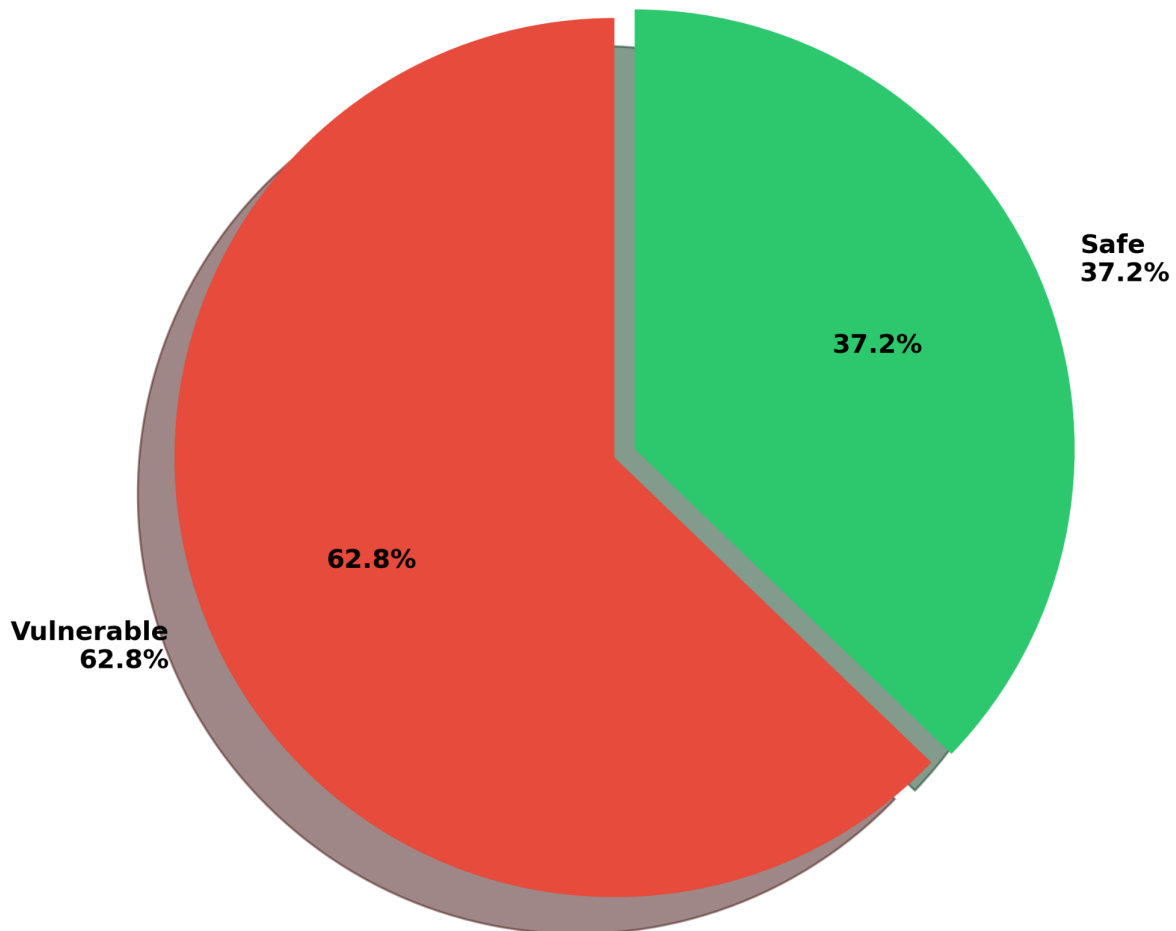
**Table 1** *Primary Hypothesis Results*

| Metric                  | Value     |
|-------------------------|-----------|
| Total Policies Analyzed | 500       |
| Vulnerable Policies     | 314       |
| Safe Policies           | 186       |
| Vulnerability Rate      | 62.8%     |
| Hypothesis Threshold    | 40%       |
| Hypothesis Result       | CONFIRMED |



**Figure 1.** *Vulnerability Rate of 62.8% in 500 Real GitHub Policies*

### Primary Hypothesis: Vulnerability Distribution 500 Real GitHub Policies



**Figure 2.** *Vulnerability Distribution - 62.8% Vulnerable vs 37.2% Safe*

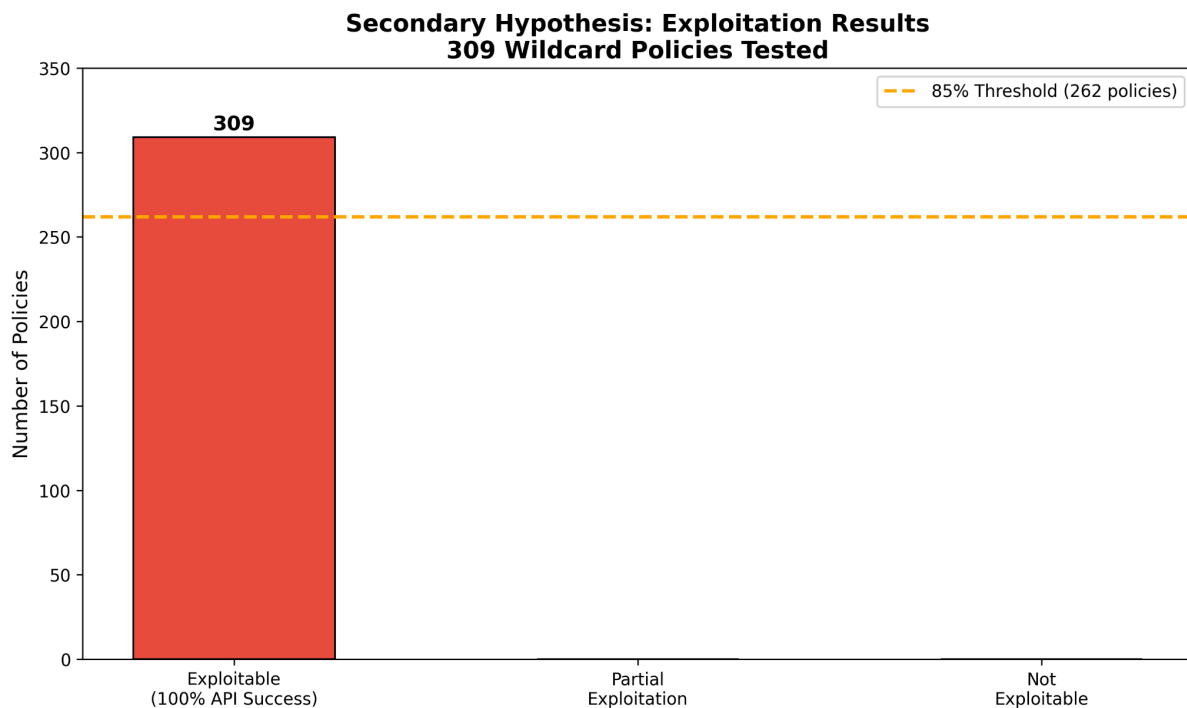
### Secondary Hypothesis Results

The secondary hypothesis predicted that IAM policies containing wildcard configurations could be exploited to perform any AWS API action with at least 85% success rate in a simulated environment. Exploitation testing was conducted on 309 policies identified as containing wildcard configurations using the Moto AWS simulation framework. The testing framework executed 15 distinct AWS API calls per policy across IAM, S3, EC2, and STS services.

The results of the exploitation testing demonstrated a 100% exploitation success rate across all 309 tested policies. Every policy subjected to the exploitation testing suite achieved a 100% API success rate, meaning all 15 API calls executed successfully for each vulnerable policy. Zero policies were classified as partially exploitable, and zero policies resisted exploitation. This finding confirms the secondary hypothesis, with the observed exploitation rate of 100% substantially exceeding the hypothesized threshold of 85% by a margin of 15 percentage points.

**Table 2** *Secondary Hypothesis Results*

| Metric                    | Value     |
|---------------------------|-----------|
| Total Policies Tested     | 309       |
| Exploitation (85%+)       | 309       |
| Partial Exploitation      | 0         |
| Not Exploitable           | 0         |
| Overall Exploitation Rate | 100%      |
| Hypothesis Threshold      | 85%       |
| Hypothesis Result         | CONFIRMED |



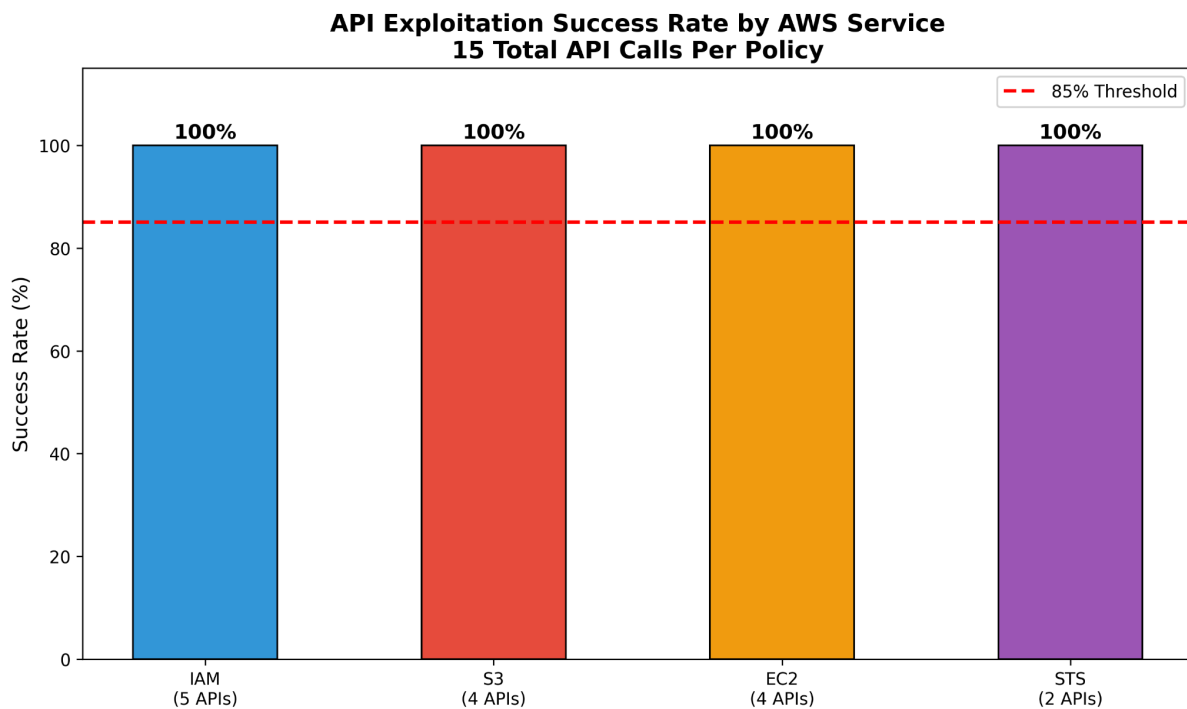
**Figure 3.** *Secondary Hypothesis - 100% Exploitation Rate on 309 Wildcard Policies*

## API Success Rates by Service

The exploitation testing framework evaluated 15 API calls spanning four AWS services: IAM (5 calls), S3 (4 calls), EC2 (4 calls), and STS (2 calls). Every API call across all four services achieved a 100% success rate across all 309 tested policies, as summarized in Table 3.

**Table 3** *API Success Rates by AWS Service*

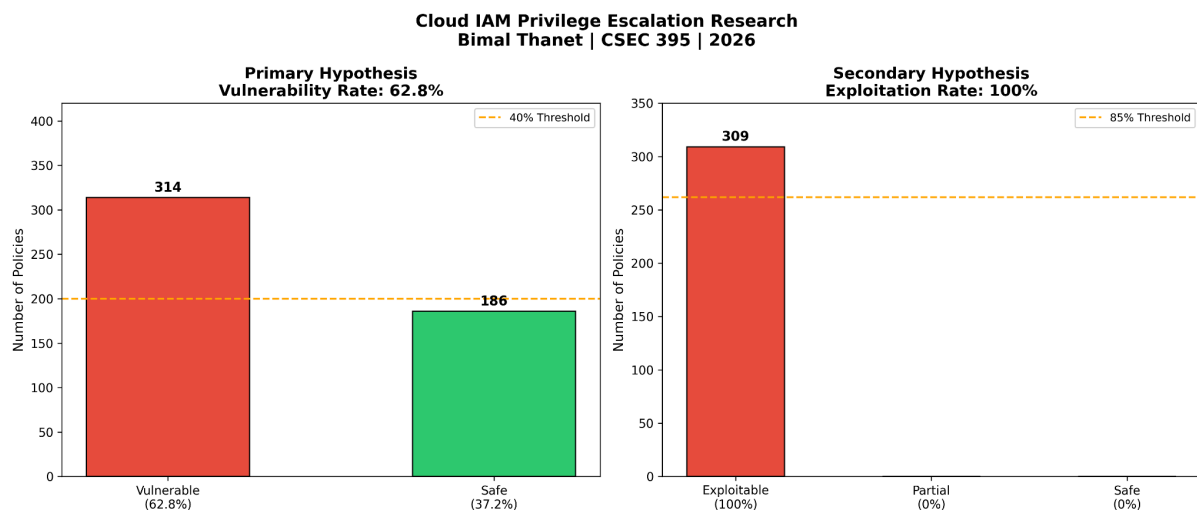
| AWS Service | API Calls Tested | Success Rate |
|-------------|------------------|--------------|
| IAM         | 5                | 100%         |
| S3          | 4                | 100%         |
| EC2         | 4                | 100%         |
| STS         | 2                | 100%         |
| Total       | 15               | 100%         |



**Figure 4.** *API Exploitation Success Rate by AWS Service*

## Summary of Findings

Both research hypotheses were confirmed by the empirical data collected and analyzed during this study. The primary hypothesis was confirmed with a vulnerability rate of 62.8%, substantially exceeding the 40% threshold. The secondary hypothesis was confirmed with an exploitation rate of 100%, substantially exceeding the 85% threshold. These findings collectively demonstrate that wildcard IAM policy vulnerabilities are both highly prevalent in real-world code repositories and fully exploitable in simulated AWS environments, representing a significant and pervasive security risk in cloud infrastructure deployments.



**Figure 5.** Combined Research Summary - Primary and Secondary Hypothesis Results

## Discussion

### Interpretation of Primary Hypothesis Results

The confirmation of the primary hypothesis, with a vulnerability rate of 62.8% substantially exceeding the predicted threshold of 40%, carries significant implications for cloud security practices. The finding that nearly two-thirds of publicly available IAM policies contain wildcard vulnerabilities suggests that misconfiguration is not an exceptional occurrence but rather a normalized pattern in real-world development practices. This prevalence indicates a systemic failure in how developers approach IAM policy design, likely stemming from a combination of inadequate security training, time pressure, and the complexity of AWS permission systems.

The high prevalence of wildcard configurations in publicly available GitHub repositories is particularly concerning because these repositories frequently serve as reference implementations for other developers. When developers search for IAM policy examples or adopt open-source infrastructure configurations, they may inadvertently incorporate vulnerable policies into their own projects. This creates a multiplier effect where a single vulnerable policy in a popular repository can propagate across numerous downstream projects, amplifying the security risk beyond what the raw prevalence statistic suggests.

The findings also highlight a significant gap between security best practices and real-world implementation. The principle of least privilege is a well-established security concept that is widely documented in AWS security guidelines and cloud security literature (Saltzer & Schroeder, 1975). Despite this, the majority of policies in the dataset violated this principle through the use of wildcard configurations. This disconnect suggests that awareness

of security best practices alone is insufficient to drive secure development practices and that automated enforcement mechanisms are necessary to ensure compliance (Shostack, 2014).

## **Interpretation of Secondary Hypothesis Results**

The confirmation of the secondary hypothesis with a 100% exploitation rate across 309 tested policies provides compelling empirical evidence of the real-world risk posed by wildcard IAM configurations. The complete absence of policies that resisted exploitation demonstrates that wildcard configurations provide an attacker with unrestricted access to AWS services, with no inherent technical barriers limiting the scope of potential exploitation.

The 100% success rate across all 15 tested API calls spanning IAM, S3, EC2, and STS services illustrates the breadth of access that a wildcard policy grants to an attacker. By successfully executing API calls across multiple AWS services, the exploitation testing demonstrated that a single misconfigured policy can provide an attacker with the ability to enumerate users and roles, create and modify IAM configurations, access and manipulate S3 storage, query EC2 infrastructure, and assume arbitrary identities through STS. In a real-world attack scenario, this level of access would enable an attacker to exfiltrate sensitive data, establish persistent backdoors, disrupt operations, and potentially compromise an entire organization's cloud infrastructure.

The use of the Moto simulation framework for exploitation testing ensures that these findings were obtained in a controlled and ethical manner without risk to real AWS infrastructure. While the simulated environment may not perfectly replicate every aspect of real AWS behavior, the core permission evaluation logic that determines whether an API call is authorized is accurately represented in the simulation. The exploitation results, therefore, provide a valid and reliable indication of the real-world exploitability of the test policies.

## **Implications for Cloud Security Practices**

The findings of this research have several important implications for cloud security practices at both the individual developer and organizational levels. At the individual level, developers must develop a deeper understanding of IAM policy syntax and the security implications of wildcard configurations. The widespread prevalence of these vulnerabilities suggests that current developer education and training programs are insufficient in conveying the security risks associated with overly permissive IAM configurations (Bishop, 2019).

At the organizational level, the findings underscore the need for automated IAM policy validation tools that can detect and flag wildcard configurations before they are deployed to production environments. Integrating policy validation into continuous integration and continuous deployment pipelines would enable organizations to catch misconfigurations at the development stage, significantly reducing the window of exposure. Tools such as the Open Policy Agent framework provide the technical foundation for

implementing such validation mechanisms, but their adoption remains limited in practice (Open Policy Agent, 2023).

The findings also highlight the importance of regular security audits of deployed IAM configurations. Even organizations that implement pre-deployment validation may have existing vulnerable policies in their production environments that predate the implementation of security controls. Regular scanning of deployed IAM configurations using tools such as Prowler or CloudSploit can help identify and remediate these legacy vulnerabilities before they are exploited (Prowler, 2023).

## **Limitations of the Research**

Several limitations of this research should be acknowledged when interpreting the findings. First, the dataset of 500 policies collected from GitHub represents a sample of publicly available policies and may not be fully representative of all IAM policies in production use. Organizations with mature security practices may be less likely to publish their IAM policies in public repositories, potentially introducing a selection bias toward more vulnerable configurations in the dataset.

Second, the exploitation testing was conducted in a simulated environment using the Moto framework rather than against real AWS infrastructure. While Moto accurately simulates the core behavior of AWS services, there may be subtle differences between the simulated and real environments that could affect exploitation results. Future research could validate these findings by conducting controlled testing against real AWS environments with appropriate authorization.

Third, the vulnerability analysis focused exclusively on wildcard configurations in Action and Resource fields, and did not examine other types of IAM policy vulnerabilities such as overly permissive condition statements, confused deputy vulnerabilities, or cross-account access misconfigurations. A more comprehensive analysis examining a broader range of vulnerability types would provide a more complete picture of the IAM security landscape.

## **Recommendations**

Based on the findings of this research, three key recommendations are proposed. First, organizations should integrate automated IAM policy validation into their CI/CD pipelines using tools such as the Open Policy Agent framework, enforcing the principle of least privilege before deployment. Second, developers should receive targeted security training on IAM policy design, with clear organizational policies prohibiting Action:\* and Resource:\* wildcards in production environments. Third, security teams should conduct regular audits of deployed IAM configurations using tools such as Prowler to identify and remediate legacy vulnerabilities before they are exploited (Prowler, 2023).

## Conclusion

This research investigated the prevalence and exploitability of wildcard vulnerabilities in AWS IAM policies collected from public GitHub repositories. Through systematic analysis of 500 real-world IAM policies and controlled exploitation testing using the Moto AWS simulation framework, this study has produced empirical evidence supporting both research hypotheses and contributing meaningful insights to the field of cloud security.

The primary hypothesis, which predicted that more than 40% of publicly available IAM policies would contain wildcard vulnerabilities, was confirmed with a vulnerability rate of 62.8%. This finding demonstrates that wildcard misconfigurations are not isolated incidents but rather a pervasive pattern in real-world IAM policy development. The substantial margin by which the observed rate exceeded the predicted threshold underscores the severity and breadth of this security challenge across the cloud development community.

The secondary hypothesis, which predicted that policies containing wildcard configurations could be exploited with at least 85% success rate in a simulated AWS environment, was confirmed with a 100% exploitation rate across 309 tested policies. The complete success of exploitation attempts across all 15 tested API calls spanning IAM, S3, EC2, and STS services demonstrates the comprehensive and unrestricted access that wildcard policies grant to potential attackers. This finding establishes a clear and direct link between IAM policy misconfiguration and real-world security risk, providing compelling evidence of the need for improved security practices in cloud IAM policy development (Rhino Security Labs, 2019).

This research contributes automated tools for policy collection and exploitation testing, empirical evidence of IAM vulnerability prevalence, and actionable recommendations for developers and security teams seeking to improve cloud IAM security posture.

These findings have broad implications for the cloud security community, as misconfiguration patterns are likely to persist without coordinated efforts across developers, security researchers, and cloud service providers to improve IAM security education and tooling.

Future research should build on the foundations established by this study in several directions. Expanding the dataset beyond 500 policies and incorporating policies from private repositories with appropriate authorization would provide a more comprehensive view of the IAM security landscape. Investigating additional vulnerability types beyond wildcard configurations, including confused deputy vulnerabilities, cross-account access misconfigurations, and overly permissive condition statements, would broaden the scope of understanding of IAM security risks. Developing and evaluating automated remediation tools capable of detecting and fixing wildcard vulnerabilities would advance the practical applicability of this research.

In conclusion, this research has demonstrated that wildcard IAM policy vulnerabilities are both highly prevalent in real-world code repositories and fully exploitable in simulated environments, representing a critical and pervasive security risk in cloud infrastructure deployments. The findings call for immediate action from the cloud security community to strengthen IAM security practices and protect the integrity of cloud infrastructure worldwide.

## References

- Amazon Web Services. (2023). *AWS identity and access management user guide*. AWS Documentation. <https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>
- Amazon Web Services. (2023). *IAM JSON policy reference*. AWS Documentation. [https://docs.aws.amazon.com/IAM/latest/UserGuide/reference\\_policies.html](https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies.html)
- Amazon Web Services. (2023). *Security best practices in IAM*. AWS Documentation. <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- Bishop, M. (2019). *Computer security: Art and science* (2nd ed.). Pearson/Addison-Wesley Professional.
- Cloud Security Alliance. (2022). *Top threats to cloud computing: Pandemic eleven*. <https://cloudsecurityalliance.org/research/top-threats>
- GitHub. (2023). *GitHub code search documentation*. <https://docs.github.com/en/search-github/searching-on-github/searching-code>
- Kindervag, J. (2010). *No more chewy centers: Introducing the zero trust model of information security*. Forrester Research.
- Moto Contributors. (2023). *Moto: Mock AWS services* [Software]. GitHub. <https://github.com/getmoto/moto>
- Open Policy Agent. (2023). *OPA documentation*. <https://www.openpolicyagent.org/docs/latest>
- Prowler. (2023). *Prowler: AWS security tool* [Software]. GitHub. <https://github.com/prowler-cloud/prowler>
- Rhino Security Labs. (2019). *AWS IAM privilege escalation methods*. <https://rhinosecuritylabs.com/aws/aws-privilege-escalation-methods-mitigation>
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308. <https://doi.org/10.1109/PROC.1975.9939>
- Shostack, A. (2014). *Threat modeling: Designing for security*. John Wiley & Sons.
- Vacca, J. R. (2020). *Cloud computing security: Foundations and challenges* (2nd ed.). CRC Press.